Advanced Algorithms

Lecture Notes

Luca Simonetti

2025–2026

Contents

Chapter 1

String Matching

1.1 Introduction & Exact Pattern Matching

1.1.1 Definition

The exact pattern matching problem is defined as follows:

> **Problem 1.1.1** (Exact pattern matching)**.** Given an alphabet Σ, and two strings T, $P \in \Sigma^*$, for which $|T| = n$ and $|P| = m$, find all occurrences of P in T, i.e. compute $\{\, i \mid T[i,\, i+m-1] = P \,\}$.

Notation

Throughout these notes, Σ denotes a finite alphabet of constant size ($|\Sigma| \in \mathcal{O}(1)$). For a string S of length $|S|$:

- $S[i]$ denotes the i-th character of S (1-based indexing).
- $S[i,j]$ denotes the substring $S[i]\,S[i+1]\cdots S[j]$ (empty string if $i > j$).
- $S[,j] = S[1,j]$ is the prefix of length j; $S[i,] = S[i,|S|]$ is the suffix starting at i.

A *proper* prefix (resp. suffix) of S is any prefix (resp. suffix) strictly shorter than S itself.

> **Example 1.1.2.** Let $\Sigma = \{a, b\}$, $T =$ `ababbab` and $P =$ `abb`. The only occurrence of P in T is at position 3.

1.1.2 Naive approach

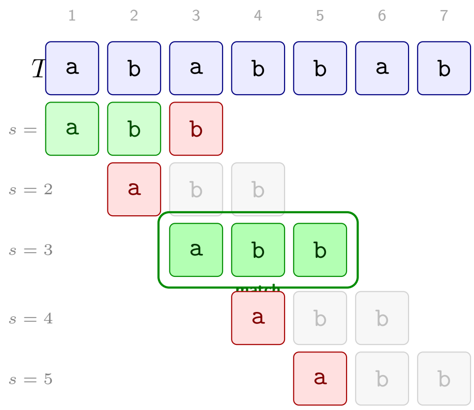


Figure 1.1: Naive exact pattern matching of $P =$ `abb` in $T =$ `ababbab` (1-indexed). Each row corresponds to one shift s; the pattern P is aligned starting at $T[s]$. **Green** = character matched, **red** = first mismatch (algorithm skips to next shift), gray = not compared.

What the Figure 1.1 illustrates is one basic naive approach. In this approach we nest two loops: the outer loop iterates over all possible "first positions" s in T where the pattern P could potentially match, and the inner loop checks character by character if P matches T starting from position s. For example the first row of the figure corresponds to starting the search from index $s = 1$ of T. Given s we then loop one by one each character of P, moving along T as well, and check if all the characters match. In this case we find a mismatch at the third character of P (which is `b`, whereas the corresponding character in T is `a`). When this happens, we break this inner loop and move to the next shift $s = 2$. If the length of the pattern was m and the length of the text was n, the outer loop could have up to $\mathcal{O}(n)$ iterations, and each inner iteration could take up to $\mathcal{O}(m)$ time, leading to a worst-case time complexity of $\mathcal{O}(nm)$ for this naive approach.

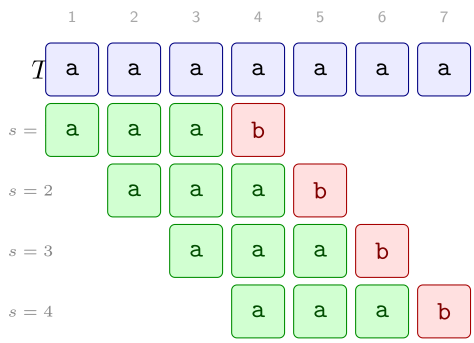


Figure 1.2: Worst-case input for the naive algorithm: $T =$ `aaaaaaa` ($n = 7$) and $P =$ `aaab` ($m = 4$). Every shift s performs $m - 1$ successful comparisons before the inevitable mismatch on $P[m] =$ `b`, giving $(n - m + 1)(m - 1) = \Theta(nm)$ total comparisons.

In figure 1.2 we see an example of a worst-case input for the naive algorithm. In this case the text T consists of n `a` characters, and the pattern P consists of $m - 1$ `a` characters followed by a `b`. There are $n - m + 1$ possible shifts s where the pattern could potentially match, and for each of these shifts the algorithm will successfully match the first $m-1$ characters of P. In the example of the figure we have $n = 7$ and $m = 4$, so there are $7 - 4 + 1 = 4$ shifts to check, and for each shift we make m checks, of which $m - 1 = 3$ are successful matches before we encounter the mismatch at the last character of P. This results in a total of $(n - m + 1)(m - 1)$ comparisons, which is $\Theta(nm)$ in the worst case.

Algorithm 1: Naive exact pattern matching

Input: Text $T[1 \dots n]$, Pattern $P[1 \dots m]$
Output: All positions s such that $T[s, s+m-1] = P$

for $s \leftarrow 1$ **to** $n - m + 1$ **do**
    $j \leftarrow 1$
    while $j \leq m$ ***and*** $T[s + j - 1] = P[j]$ **do**
        $j \leftarrow j + 1$
    if $j > m$ **then**
        output s

Correctness

To pin down what "the inner loop does" precisely, let's give a name to the obvious quantity: for any position i in T, write

$$\mathrm{common}(i) \;=\; \max\{\, k \geq 0 \mid T[i, i+k-1] = P[1,k] \,\}$$

for the length of the longest common prefix of $T[i,]$ and P — basically, how many characters of P would match if you aligned it at position i.

Lemma 1.1.3. *At shift s, the inner loop exits with $j = \mathrm{common}(s) + 1$.*

Proof. Track the invariant: entering each iteration, we have $T[s, s+j-2] = P[1, j-1]$ (the first $j-1$ characters already matched). The loop keeps going as long as the next character also matches; it stops the moment it doesn't, or when j runs past m. Since $\mathrm{common}(s)$ is exactly how many consecutive characters agree, the loop stops right at $j = \mathrm{common}(s) + 1$. □

Theorem 1.1.4. *The naive algorithm finds every occurrence of P in T, and nothing more.*

Proof. The outer loop visits every shift $s \in \{1, \dots, n-m+1\}$, so no candidate starting position is ever skipped. By Lemma 1.1.3, after the inner loop we have $j = \mathrm{common}(s) + 1$; the algorithm outputs s exactly when $j > m$, i.e. $\mathrm{common}(s) \geq m$, i.e. $T[s, s+m-1] = P$ — which is precisely the definition of an occurrence. □

Lower bound

Theorem 1.1.5. *Any algorithm for exact pattern matching must make $\Omega(n+m)$ character comparisons in the worst case.*

Proof. Two independent arguments, one for each term.

- **You have to read all of** P ($\Omega(m)$). Suppose the algorithm skips position $P[q]$. An adversary flips just that character, producing a different pattern P'. The algorithm sees the same comparisons on both inputs and therefore gives the same output — which can't be correct for both P and P'.

- **You have to read all of** T ($\Omega(n)$). Suppose position $T[i]$ is never read. The adversary flips that character, potentially creating or destroying a match right around position i. Since the algorithm never looked at $T[i]$, it has no way to tell the difference.

The two bounds add up: $\Omega(m) + \Omega(n) = \Omega(n + m)$. □

The naive algorithm achieves $\mathcal{O}(nm)$ in the worst case, which is far from this lower bound. KMP will close the gap, achieving $\Theta(n + m)$.

This $\Omega(n+m)$ lower bound applies to any *algorithm, not just naive.*

Properties

We evaluate string matching algorithms along four axes:

Online. The algorithm processes T strictly left-to-right, one character at a time, and can emit results before reading future characters of T. Useful when T arrives as a stream.

Real-time. A stronger requirement: each new character of T is fully processed in $\mathcal{O}(1)$ *worst-case* time before the next one is read.

Blind (no backtracking). The algorithm never re-reads a position of T: each character of T is accessed at most once. A blind algorithm is necessarily online; the converse is false (the naive algorithm is online but not blind).

Succinct. The algorithm uses only $\mathcal{O}(1)$ extra space beyond storing P and T (no large auxiliary data structures).

Algorithm	Online	Real-time	Blind	Succinct
Naive	✓	×	×	✓
KMP	✓	×	✓	×
KMP (sp')	✓	✓	✓	×

The naive algorithm backtracks in T: at each new shift s it re-reads characters already compared at shift $s-1$.

The naive algorithm is **online** (it reads T left-to-right) but not **blind**: at each new shift it backtracks and re-compares characters of T that were already seen. It is also not **real-time**: a single character of T can trigger up to $\mathcal{O}(m)$ comparisons.

This naive approach basically starts over from scratch at each shift, without leveraging precious information from previous comparisons.

1.1.3 Knuth-Morris-Pratt (KMP) algorithm

Intuition

What if we could somehow "remember" the information from a previous comparison, and use it to skip some of the checks at the next shift? Let's introduce the intuition behind the Knuth-Morris-Pratt (KMP) algorithm, with an example.

Example 1.1.6. Let $P =$ `abcdeabdef` and suppose we are running the naive algorithm at shift $s = 3$ over some text T whose characters at positions 3–9 are `abcdeab`. We match $P[1\dots7]$ successfully, then hit a mismatch at $P[8] =$ `d` vs $T[10] =$ `x`.

As shown in Figure 1.3, at this point the naive algorithm would discard everything and try shift $s = 4$. But notice that the matched portion `abcdeab` starts *and* ends with $\alpha =$ `ab`: the prefix $P[1\dots2]$ and the suffix $P[6\dots7]$ of the matched region are identical. This means that when we slide P forward, the α that was at the end of the match is now perfectly aligned with the α at the start of P — we can jump directly to shift $s = 8$ and resume comparing from $P[3]$, skipping five shifts entirely.

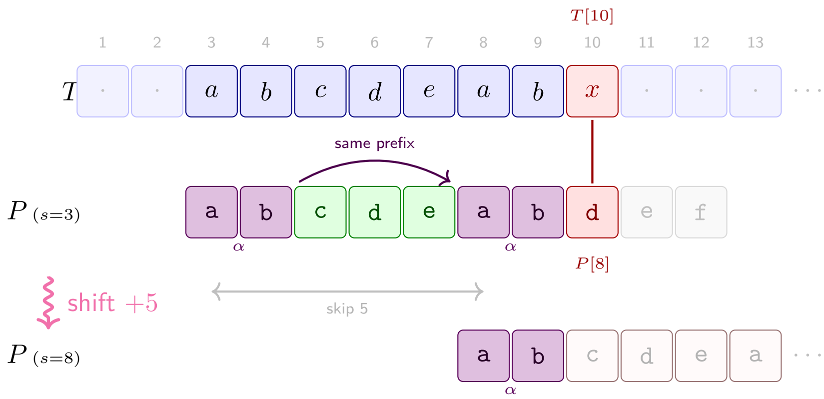
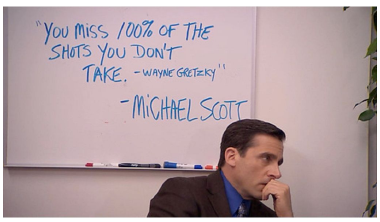


Figure 1.3: KMP shift intuition with $P =$ `abcdeabdef`. At shift $s = 3$, the algorithm matches $P[1\dots7] =$ `abcdeab` then hits a mismatch at $P[8]$. The matched portion ends with $\alpha =$ `ab`, which also appears at the very start of P — so instead of shifting by 1, we jump directly to $s = 8$, re-aligning α and resuming from $P[3]$.

Basically what the example shows is that if we take the longest prefix of P that is also a suffix of the **matched** portion of P, we can shift the pattern so that this prefix is now aligned with the suffix — and we can resume the search from the character right after this prefix, since we know it must match (otherwise we would have had a mismatch earlier). In other words we have saved some work (namely, five shift indexes) by leveraging the information from the previous comparisons, instead of starting over from scratch, from the very next shift.

When we say suffix here we mean suffix of the ***matched*** *portion of P, not of P itself. In the example, α is a suffix of $P[1\dots7]$ but not of the whole P.*

But is this correct? Or do we miss some potential matches by jumping directly to $s = 8$?



"You miss 100% of the shots you don't take." — Michael Scott

Well, it turns out that this is indeed correct, and we won't miss any matches by doing this. The KMP algorithm is based on this very idea, and it guarantees that we will find all occurrences of P in T without missing any, while also achieving a much better time complexity than the naive approach.

Preprocessing: computing *sp* values

To exploit this idea, we need to precompute for every prefix $P[1 \dots i]$ its *longest proper prefix that is also a suffix*. We write this length as $\mathrm{sp}_i(P)$.

A proper prefix/suffix excludes the string itself; $\mathrm{sp}_i(P)$ *is thus an integer.*

> **Definition 1.1.7** (*sp* values)**.** Let P be a pattern of length m. For each $i \in \{1, \dots, m\}$, define
>
> $$\mathrm{sp}_i(P) \;=\; \max\{\, k < i \mid P[1 \dots k] = P[i-k+1 \dots i] \,\},$$
>
> i.e. the length of the longest proper prefix of $P[1 \dots i]$ that is also a suffix of $P[1 \dots i]$.

The key observation is that $\mathrm{sp}_i(P)$ can be computed recursively from $\mathrm{sp}_{i-1}(P)$, giving an efficient $\mathcal{O}(m)$ preprocessing algorithm. There are two cases:

1. **Extension is possible.** Let $k = \mathrm{sp}_{i-1}(P)$. If $P[k+1] = P[i]$, then the longest prefix-suffix of $P[1 \dots i]$ is simply one character longer than that of $P[1 \dots i-1]$, so $\mathrm{sp}_i(P) = k+1$.

   Example: with $P =$ `abcdeabdef` at $i = 7$, we have $\mathrm{sp}_6(P) = 1$ (the prefix-suffix of `abcdea` is `a`, length 1). We check $P[2] =$ `b` against $P[7] =$ `b`: they match, so $\mathrm{sp}_7(P) = 2$ (see Figure 1.4).

2. **Extension is not possible.** If $P[k+1] \neq P[i]$, we cannot extend the current prefix-suffix. We then ask: what is the longest prefix-suffix of the prefix-suffix itself? In other words, we fall back to $k \leftarrow \mathrm{sp}_k(P)$ and try to extend that instead. We repeat this process until either a match is found or we reach $k = 0$, in which case $\mathrm{sp}_i(P) = 0$.

   Example: still with $P =$ `abcdeabdef` at $i = 8$, we have $\mathrm{sp}_7(P) = 2$ (the prefix-suffix of `abcdeab` is `ab`). We check $P[3] =$ `c` against $P[8] =$ `d`: no match. We fall back to $k \leftarrow \mathrm{sp}_2(P) = 0$, so we check $P[1] =$ `a` against $P[8] =$ `d`, still no match. We have reached $k = 0$, so $\mathrm{sp}_8(P) = 0$.

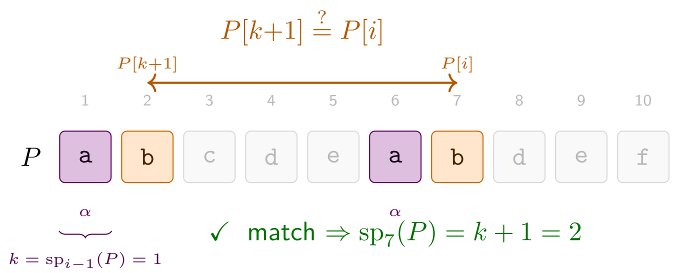


Figure 1.4: Computing $\mathrm{sp}_7(P)$ for $P =$ `abcdeabdef`. We know $k = \mathrm{sp}_6(P) = 1$: the suffix of $P[1 \dots 6]$ matching a prefix of P has length 1 (namely $\alpha =$ `a`, violet). We compare $P[k+1] = P[2] =$ `b` against $P[i] = P[7] =$ `b` (orange): they match, so $\mathrm{sp}_7(P) = 2$.

The resulting algorithm is the following:

Algorithm 2: Computing sp values

Input: Pattern $P[1 \dots m]$
Output: Array $sp[1 \dots m]$ where $sp[i] = \mathrm{sp}_i(P)$

```
sp[1] ← 0
k ← 0                // length of current candidate prefix-suffix
for i ← 2 to m do
    while k > 0 and P[k+1] ≠ P[i] do
        k ← sp[k]                                   // fall back
    if P[k+1] = P[i] then
        k ← k + 1
    sp[i] ← k
return sp
```

Remark 1.1.8 (Why fall back to $sp[k]$?). At every point in the loop, k has a precise meaning: it is the length of the longest prefix of P that currently matches a suffix of $P[1 \dots i{-}1]$. In other words,

$$P[1 \dots k] \;=\; P[i-k \dots i-1],$$

the k characters immediately before position i are exactly the first k characters of the pattern. When $P[k+1] = P[i]$ we can extend this match by one: $sp[i] = k+1$. When $P[k+1] \neq P[i]$ the match of length k cannot be extended, and we need a shorter candidate $j < k$ that still satisfies the same property: $P[1 \dots j] = P[i-j \dots i-1]$.

Key insight. $P[1 \dots k]$ is not an arbitrary string — it is a *prefix* of P. Any valid candidate j must be a prefix of P that also appears as a suffix of $P[1 \dots i{-}1]$. Two simple observations handle this:

- *Prefix of a prefix is a prefix*: if $P[1 \dots j]$ is a prefix of $P[1 \dots k]$, it is a prefix of P.
- *Suffix of a suffix is a suffix*: since $P[1 \dots k]$ is a suffix of $P[1 \dots i{-}1]$ (by the invariant), any suffix of $P[1 \dots k]$ of length j is also a suffix of $P[1 \dots i{-}1]$.

Therefore j is a valid candidate *if and only if* $P[1 \dots j]$ is a *border* of $P[1 \dots k]$ (a string that is both a prefix and a suffix of it). The longest such border is $sp[k]$, already computed during preprocessing. Any value strictly between $sp[k]$ and k is, by definition of $sp[k]$, not a border of $P[1 \dots k]$ and therefore not a valid candidate — no check is needed. We jump directly to $k \leftarrow sp[k]$ and repeat. Figure 1.5 shows this chain for $i = 6$, $k = 3$.

Note 1.1.9 (A fallback that succeeds). Let $P =$ `ababcababa` and consider $i = 10$. We have $k = 4$ since $P[1 \dots 4] = P[6 \dots 9] =$ `abab`. Check $P[5] =$ `c` vs. $P[10] =$ `a`: mismatch. Could $k' = 3$ be a valid candidate? It would need $P[1 \dots 3]$ to be a border of `abab`, i.e. `aba` $=$ `bab`: no. The longest border of `abab` is `ab` (length 2), so $sp[4] = 2$. We fall back to $k = 2$: $P[1 \dots 2] = P[8 \dots 9] =$ `ab`. Check $P[3] =$ `a` vs. $P[10] =$ `a`: **match**! Hence $sp[10] = 3$; skipping $k' = 3$ was the only correct move.

Formal details — **Why no candidate between $sp[k]$ and k exists**

Suppose for contradiction that $sp[k] < k' < k$ and that k' satisfies the same property as k, i.e. $P[1 \dots k'] = P[i - k' \dots i - 1]$. Since $k' < k$, the string $P[i - k' \dots i - 1]$ is a suffix of $P[i - k \dots i - 1] = P[1 \dots k]$. Combined with $P[1 \dots k']$ being a prefix of P, this makes $P[1 \dots k']$ a border of $P[1 \dots k]$ of length $k' > sp[k]$, contradicting $sp[k]$ being the *longest* proper border of $P[1 \dots k]$. $\square$

Formal details — **Correctness of the sp preprocessing algorithm**

We prove that at the end of iteration i, $sp[i]$ equals the length of the longest proper border of $P[1 \dots i]$.

Precondition	$P[1 \dots m]$ is given; $sp[1] = 0$ and $k = 0$ are set before the loop.
Loop guard	i ranges from 2 to m.
Invariant	At the *start* of iteration i: $k = sp[i-1]$, i.e. k is the length of the longest proper border of $P[1 \dots i-1]$ (and in particular $P[1 \dots k] = P[i-k \dots i-1]$).
Postcondition	$sp[i]$ is correctly set for all $1 \le i \le m$.

Base case	Before the first iteration ($i = 2$), $k = 0 = sp[1]$ since $P[1 \dots 1]$ has no proper border. ✓
Preservation	Assume the invariant holds at the start of iteration i, i.e. $k = sp[i-1]$. The inner **while** loop traverses the border chain $k \to sp[k] \to sp[sp[k]] \to \cdots$, stopping as soon as $P[k+1] = P[i]$ or $k = 0$. By the border characterisation (a candidate j is valid iff $P[1 \dots j]$ is a border of $P[1 \dots k]$), this finds the longest extendable border. The subsequent **if** either increments k by one or leaves $k = 0$. In both cases $sp[i] \leftarrow k$ is correct, and $k = sp[i]$ is the invariant for iteration $i+1$. ✓
Postcondition	When the loop exits ($i = m + 1$), all $sp[i]$ have been correctly computed. ✓

Step-by-step: Computing sp values

Let's trace the algorithm on $P = \texttt{abababaca}$.

1. $i = 1$**:** By definition $sp[1] = 0$. $k \leftarrow 0$.
2. $i = 2$**:** $k = 0$, $P[1] = \texttt{a} \neq P[2] = \texttt{b}$. $sp[2] = 0$.
3. $i = 3$**:** $k = 0$, $P[1] = \texttt{a} = P[3] = \texttt{a}$. $k \leftarrow k + 1 = 1$. $sp[3] = 1$.
4. $i = 4$**:** $k = 1$, $P[2] = \texttt{b} = P[4] = \texttt{b}$. $k \leftarrow k + 1 = 2$. $sp[4] = 2$.
5. $i = 5$**:** $k = 2$, $P[3] = \texttt{a} = P[5] = \texttt{a}$. $k \leftarrow k + 1 = 3$. $sp[5] = 3$.
6. $i = 6$**:** $k = 3$. The invariant gives $P[1 \dots 3] = P[3 \dots 5] = \texttt{aba}$. We check $P[4] = \texttt{b}$ against $P[6] = \texttt{c}$: mismatch, so we fall back. The fallback chain is shown in Figure 1.5:
    - $k \leftarrow sp[3] = 1$: the longest border of $\texttt{aba}$ is $\texttt{a}$ (length 1). Check $P[2] = \texttt{b}$ against $P[6] = \texttt{c}$: mismatch.

- $k \leftarrow sp[1] = 0$: the longest border of `a` is empty. Check $P[1] =$ `a` against $P[6] =$ `c`: mismatch.
- $k = 0$: no candidate left. $sp[6] = 0$.

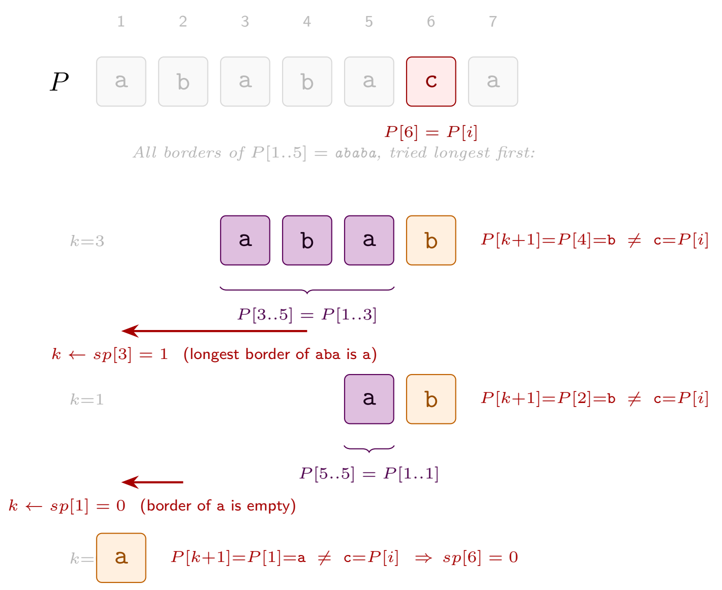


Figure 1.5: Fallback chain at step $i = 6$ for $P =$ `ababaca`. The three rows show *all* borders of $P[1..5] =$ `ababa` (lengths 3, 1, 0), tried longest first. For each candidate k, the violet cells show the matched suffix $P[i-k..i-1] = P[1..k]$; the orange cell is the character $P[k+1]$ we would need at position i to extend it. All three fail, so $sp[6] = 0$.

7. $i = 7$**:** $k = 0$, $P[1] =$ `a` $= P[7] =$ `a`. $k \leftarrow k + 1 = 1$. $sp[7] = 1$.

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$sp[i]$	0	0	1	2	3	0	1

Remark 1.1.10. The "fall back to the *sp* value of the current *sp* value" is the recursive heart of KMP: the same shift logic used during the search phase is also used during preprocessing.

Exercise 1.1.11. Design an algorithm that, given a pattern P of length m, computes $\mathrm{sp}_i(P)$ for every $i \in \{1, \ldots, m\}$ in $\mathcal{O}(m)$ time.

Hints:

1. Try to express $\mathrm{sp}_i(P)$ in terms of $\mathrm{sp}_{i-1}(P)$. What is the relationship between the longest prefix-suffix of $P[1 \ldots i]$ and that of $P[1 \ldots i-1]$?
2. There are two cases depending on whether $P[\mathrm{sp}_{i-1}(P) + 1]$ equals $P[i]$ or not. What can you do in each case?
3. For the harder case, think recursively: if you cannot extend a prefix-suffix of length k, which shorter candidate should you try next?

Exercise 1.1.12 (Period of a string). The *period* of a string S of length n is the smallest integer $p \geq 1$ such that $S[i] = S[i+p]$ for all $1 \leq i \leq n-p$. Equivalently, p is the length of the shortest string Π such that S is a prefix of $\Pi^\infty = \Pi\Pi\Pi\Pi \cdots$

Design an $\mathcal{O}(n)$-time algorithm that, given S, computes its period.

Hints:

1. Compute the *sp* values of S.
2. What is the relationship between $sp_n(S)$ and the period of S? In particular, is $n - sp_n(S)$ always a period?
3. Is it always the *shortest* period? What additional condition on $sp_n(S)$ guarantees minimality?

Intermezzo — **Amortised analysis — the credit method**

Some algorithms contain operations that look expensive in isolation but are cheap *on average* over any sequence of calls. *Amortised analysis* makes this precise by distributing the cost of occasional expensive operations across the many cheap ones that made them possible.

A clean way to formalise this is the **credit method**.

Definition 1.1.13 (Credit method)**.** Assign each operation an *amortised cost* $\hat{c}$ (which we choose freely). When an operation with real cost c runs:

- if $\hat{c} > c$, the surplus $\hat{c} - c$ is deposited as *credits* into a bank;
- if $\hat{c} < c$, the deficit $c - \hat{c}$ is withdrawn from the bank.

Invariant: the bank balance must never go negative.

Consequence: since the bank starts at 0 and never goes below 0,

$$\sum_i c_i \leq \sum_i \hat{c}_i,$$

i.e. the total real cost is bounded by the total amortised cost.

Example: stack with push and multi-pop. Consider a stack supporting two operations:

- PUSH(x): push element x onto the stack; real cost 1.
- MULTI-POP(k): pop the top $\min(k, |S|)$ elements; real cost = number of elements actually popped.

A single MULTI-POP can cost $\Theta(n)$, so a naive worst-case analysis gives $\mathcal{O}(n^2)$ for n operations. Amortised analysis does much better.

Credit assignment. Attach 1 credit to each element at the moment it is pushed:

- PUSH: amortised cost $\hat{c} = 2$ (pay 1 for the real push, deposit 1 credit *on the element*).
- MULTI-POP(k): amortised cost $\hat{c} = 0$ (use the 1 credit stored on each popped element to pay for its removal).

The bank balance equals the number of elements on the stack, which is always ≥ 0, so the invariant holds.

Example 1.1.14 (Credit trace for a stack)**.** The table below traces the sequence PUSH(a), PUSH(b), PUSH(c), MULTI-POP(2), PUSH(d), PUSH(e), MULTI-POP(3).

Operation	Stack after	Real cost	Amortised cost	Bank
PUSH(a)	$[a]$	1	2	1
PUSH(b)	$[a, b]$	1	2	2
PUSH(c)	$[a, b, c]$	1	2	3
MULTI-POP(2)	$[a]$	2	0	1
PUSH(d)	$[a, d]$	1	2	2
PUSH(e)	$[a, d, e]$	1	2	3
MULTI-POP(3)	$[\,]$	3	0	0
Total		**10**	**10**	

The bank never goes negative, confirming the invariant. The total real cost 10 is indeed $\leq$ the total amortised cost 10.

Lemma 1.1.15. *Any sequence of n PUSH and MULTI-POP operations on an initially empty stack costs $\mathcal{O}(n)$ in total.*

Proof. There are at most n pushes, each with amortised cost 2, and all MULTI-POPs have amortised cost 0. The total amortised cost is therefore at most $2n$. By the credit-method bound, the total real cost is at most $2n = \mathcal{O}(n)$. □

Application to KMP preprocessing

The sp-computation algorithm runs in $\mathcal{O}(m)$ total time despite the nested loops. We apply the credit method with k as the potential: credits measure how much k can still decrease.

- k increases by at most 1 per outer iteration (there are $m - 1$ iterations, so k grows at most $m - 1$ times in total).
- Each execution of the inner **while** body sets $k \leftarrow sp[k] < k$, strictly decreasing k.
- $k \geq 0$ always holds.

Assign amortised cost 2 to each outer step that increments k (real cost 1 plus deposit 1 credit), and amortised cost 0 to each inner iteration that decrements k (use the saved credit). The bank balance equals $k \geq 0$, so the invariant holds, and the total number of inner iterations is at most the total number of increments, which is $\mathcal{O}(m)$. Together with the $\mathcal{O}(m)$ overhead of the outer loop, the total preprocessing time is $\mathcal{O}(m)$.

Example 1.1.16. Consider the pattern $P =$ `aaaaa` of length 5. The sp values are $sp[1] = 0$, $sp[2] = 1$, $sp[3] = 2$, $sp[4] = 3$, $sp[5] = 4$. The variable

> k starts at 0 and increases to 1, then to 2, then to 3, then to 4, as we process each character of P. There are no decreases in this case, so the inner loop runs only once per outer iteration, for a total of $\mathcal{O}(m)$ time.

The search phase

Once the *sp* array is computed, we scan T left-to-right, maintaining j: the number of characters of P currently matched.

Algorithm 3: KMP search

Input: Text $T[1 \ldots n]$, pattern $P[1 \ldots m]$, precomputed $sp[1 \ldots m]$
Output: All starting positions s where P occurs in T

```
j ← 0                                // characters of P matched so far
for i ← 1 to n do
    while j > 0 and P[j+1] ≠ T[i] do
        j ← sp[j]                    // fall back along the border chain
    if P[j+1] = T[i] then
        j ← j + 1
    if j = m then
        output i − m + 1             // occurrence starting here
        j ← sp[j]                    // continue searching
```

Figure 1.6 traces the search on a concrete example, showing how *sp* values let the algorithm carry previously matched characters across fallbacks instead of starting from scratch.

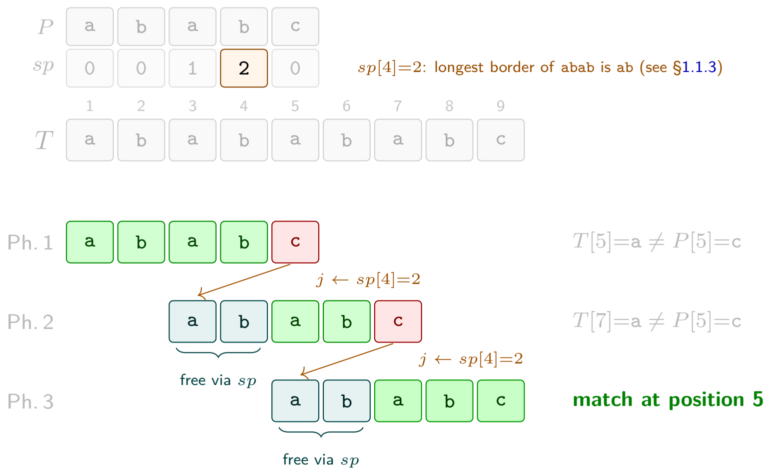


Figure 1.6: KMP search for P=`ababc` (m=5) in T=`abababab c` (n=9). The *sp* array for P is shown at the top (cf. the preprocessing in §1.1.3). Green: characters newly matched in this phase. Teal: characters carried for free from the previous phase via $j \leftarrow sp[j]$. Red: mismatch. After each fallback, the algorithm reuses the last $sp[j]$ matched characters without any comparison, then continues from where it left off in T.

The same amortised argument applies: j increases by at most 1 per outer step and each inner fallback strictly decreases $j \geq 0$, so the inner loop runs $\mathcal{O}(n)$ times in total.

Theorem 1.1.17. *The KMP algorithm finds all occurrences of P in T in $\mathcal{O}(n+m)$ time.*

1.1.4 Z-Algorithm

Definition

The Z-algorithm is a second linear-time preprocessing approach. Where $\mathrm{sp}_i(P)$ looks *backward* (longest prefix that is also a suffix of $P[1 \dots i]$), the Z-value Z_i looks *forward* from position i.

Definition 1.1.18 (Z values)**.** Let A be a string of length n. For $i \geq 2$, define

$$Z_i(A) = \begin{cases} 0 & \text{if } A[i] \neq A[1], \\ \max\{\, \ell > 0 \mid A[1,\ell] = A[i,\, i+\ell-1] \,\} & \text{otherwise.} \end{cases}$$

In words, $Z_i(A)$ is the length of the longest prefix of A that matches A starting at position i, as shown in Figure 1.7.

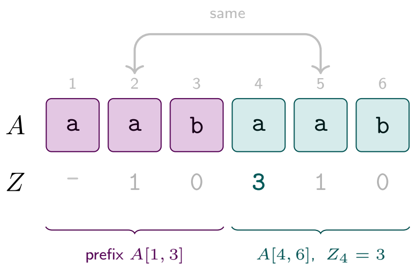


Figure 1.7: Z values for A = `aabaab` ($n = 6$). The prefix $A[1,3]$ = `aab` (violet) reappears at position 4 (teal), giving $Z_4 = 3$. Also $Z_2 = Z_5 = 1$ since $A[1] = A[2] = A[5] = \texttt{a}$, and $Z_3 = Z_6 = 0$ since $A[3] = A[6] = \texttt{b} \neq A[1]$. Z_1 is left undefined.

Connection to pattern matching. Choose a separator $\$ \notin \Sigma$ and form $A = P\$T$ (of length $m+1+n$). For any index i in the T-part of A (i.e. $i > m+1$):

$$Z_i(P\$T) = m \quad \Longleftrightarrow \quad P \text{ occurs in } T \text{ starting at position } i-m-1.$$

The separator prevents Z_i from exceeding m, so equality with m detects all occurrences.

Formal details — **Proof of the connection**

Write $A = P\$T$ and fix $i > m+1$ (i.e. i is in the T-part of A).

The separator bounds $Z_i \leq m$. If $Z_i > m$ then $A[i \dots i+m] = A[1 \dots m+1]$, so in particular $A[i+m] = A[m+1] = \$$. But $i+m > 2m+1 > m+1$, so $A[i+m]$ lies in the T-part of A, hence $A[i+m] \in \Sigma$. Since $\$ \notin \Sigma$ we have a contradiction: $Z_i \leq m$.

($\Rightarrow$) Suppose $Z_i = m$. Then $A[i \dots i+m-1] = A[1 \dots m] = P$. Since $A[i+j-1] = T[i-m-1+j-1]$ for $j = 1, \dots, m$, this gives

$T[i-m-1\dots i-1] = P$, i.e. P occurs at position $i-m-1$ in T.

($\Leftarrow$) Suppose P occurs at position $j = i-m-1$ in T, i.e. $T[j\dots j+m-1] = P$. Then $A[i\dots i+m-1] = T[j\dots j+m-1] = P = A[1\dots m]$, so $Z_i \geq m$. Combined with $Z_i \leq m$ we get $Z_i = m$. □

Example 1.1.19. Let $P =$ `aba` ($m = 3$) and $T =$ `ababac` ($n = 6$). Form $A =$ `aba$ababac` and compute Z values for positions $i > 4$:

i	1	2	3	4	5	6	7	8	9	10
$A[i]$	`a`	`b`	`a`	`$`	`a`	`b`	`a`	`b`	`a`	`c`
Z_i	–	0	1	0	**3**	0	**3**	0	1	0

The two positions with $Z_i = 3 = m$ are $i = 5$ and $i = 7$, corresponding to occurrences of P in T starting at positions $5-3-1 = 1$ and $7-3-1 = 3$. Indeed $T[1\dots 3] =$ `aba` and $T[3\dots 5] =$ `aba`. ✓

Computing Z values: the Z-box algorithm

A naive computation of all Z values costs $\mathcal{O}(n^2)$: for each i, compare character by character until a mismatch.

Intuition: reuse past work. Suppose we have processed positions $2,\dots,i-1$ and at some earlier position l we found $Z_l = z$, i.e. $A[l\dots l+z-1] = A[1\dots z]$. Call the interval

$$[l,\ r], \quad r = l+z-1$$

a *Z-box*: a contiguous block that we *know* is an exact copy of the prefix $A[1\dots z]$.

Now suppose we want Z_i and i falls *inside* the box ($l \leq i \leq r$). Let $k = i-l+1$ be the *mirror* position of i in the prefix. Since the box says $A[l\dots r] = A[1\dots r-l+1]$, we immediately know

$$A[i] = A[k], \quad A[i+1] = A[k+1], \quad \dots, \quad A[r] = A[r-l+1].$$

Whatever is already recorded in Z_k — the length of the match starting at k in the prefix — is available at i *for free*, without any character comparisons. The only open question is whether the match at i extends beyond r, which requires at most one fresh comparison per step.

The Z-box invariant. The algorithm maintains the Z-box $[l,r]$ with the **largest right endpoint** seen so far:

$$r = l + Z_l(A) - 1 \quad \text{(maximised over all processed positions).}$$

Figure 1.8 shows this setup schematically.

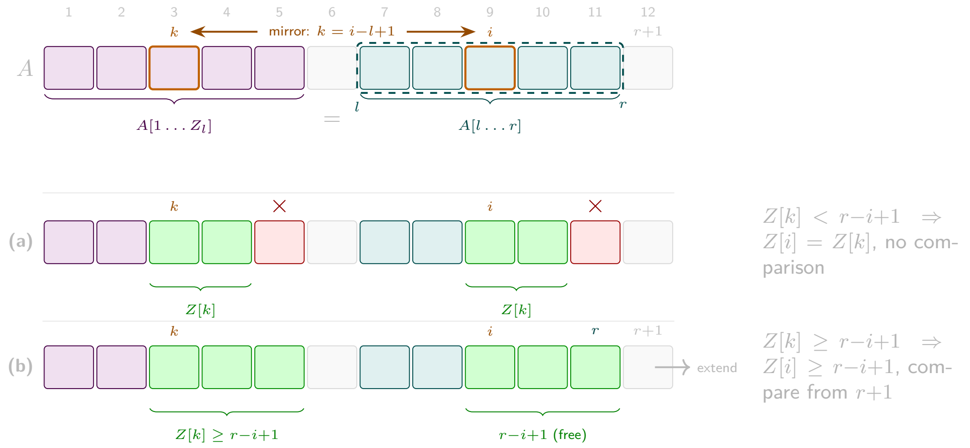


Figure 1.8: Z-box schematic (positions l=7, r=11, k=3, i=9). Violet: the prefix $A[1 \dots Z_l]$ that established the box. Teal: the Z-box $A[l \dots r]$, a copy of that prefix. Orange border: mirror pair (k, i) with $k = i{-}l{+}1$. Green: the $Z[k]$ characters that match at both k and i (free, by the mirror property). Red ×: the mismatch that terminated $Z[k]$, reflected at i. Case (a): the mismatch lies inside $[l, r]$, so $Z[i] = Z[k]$ at no cost. Case (b): the match reaches r, so we must compare from $r{+}1$. See Figure 1.9 for a concrete step-by-step trace.

At each step i (from 2 to n) exactly one of three cases applies (illustrated in Figure 1.8):

1. $i > r$ **(outside the box).** No cached information is available. Compare $A[1], A[2], \dots$ against $A[i], A[i+1], \dots$ directly until a mismatch; set Z_i to the number of matches. If $Z_i > 0$, update $l \leftarrow i,\ r \leftarrow i + Z_i - 1$.
2. $i \leq r$ **(inside the box) — let $k = i{-}l{+}1$.**
   (a) $Z_k < r - i + 1$ **(mirror strictly inside).** The Z-match at k ends before the box boundary, and the same mismatch is reflected at i: set $Z_i = Z_k$, no comparisons needed, no update to $[l, r]$.
   (b) $Z_k \geq r - i + 1$ **(mirror reaches or exceeds the boundary).** We get $Z_i \geq r{-}i{+}1$ for free; continue comparing from $A[r + 1]$ until a mismatch, then update $l \leftarrow i,\ r \leftarrow i + Z_i - 1$.

Figure 1.9 traces the *complete* execution on $A =$ `aabaaab`, showing every step $i = 2, \dots, 7$.

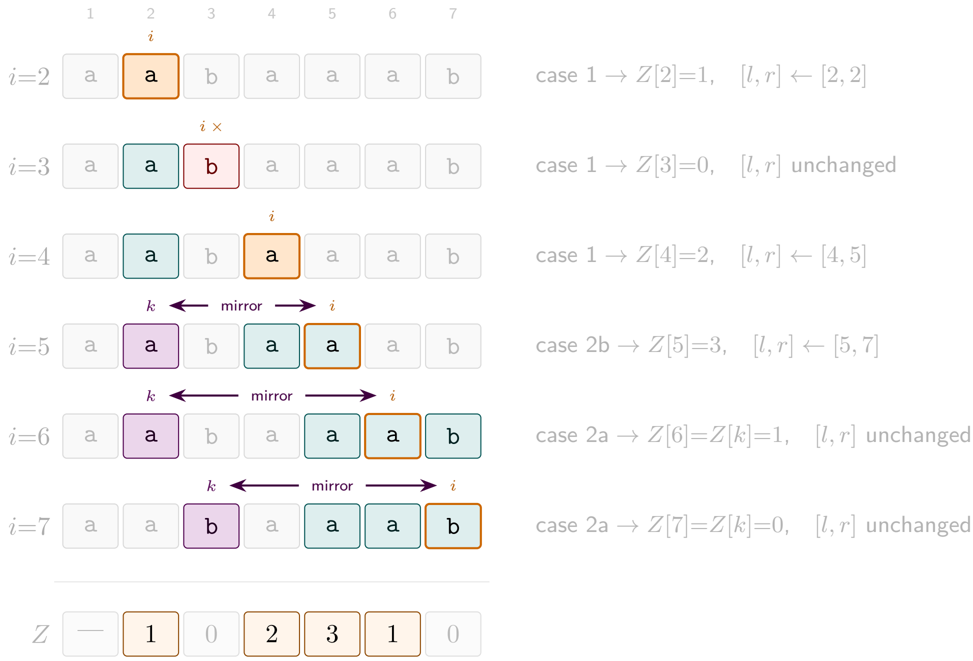


Figure 1.9: Complete Z-box trace on A = `aabaaab`, steps $i = 2, \ldots, 7$. Teal: current Z-box $[l, r]$. Orange: current position i (teal fill when i is inside the box). Violet: mirror $k = i - l + 1$. Red: immediate mismatch (Z=0). Bottom row: final Z array (non-zero entries highlighted).

Algorithm 4: Z-Algorithm

Input: String $A[1 \ldots n]$
Output: Array $Z[2 \ldots n]$ with $Z[i] = Z_i(A)$

$l \leftarrow 0$
$r \leftarrow 0$
for $i \leftarrow 2$ **to** n **do**
  if $i > r$ **then**
    $Z[i] \leftarrow 0$
    while $i + Z[i] \leq n$ ***and*** $A[1 + Z[i]] = A[i + Z[i]]$ **do**
      $Z[i] \leftarrow Z[i] + 1$
  else
    $k \leftarrow i - l + 1$
    if $Z[k] < r - i + 1$ **then**
      $Z[i] \leftarrow Z[k]$ `// fully inside - no comparisons`
    else
      $Z[i] \leftarrow r - i + 1$
      while $i + Z[i] \leq n$ ***and*** $A[1 + Z[i]] = A[i + Z[i]]$ **do**
        $Z[i] \leftarrow Z[i] + 1$
  if $Z[i] > 0$ ***and*** $i + Z[i] - 1 > r$ **then**
    $l \leftarrow i$
    $r \leftarrow i + Z[i] - 1$
return Z

FORMAL DETAILS — **Correctness of the Z-box algorithm**

We prove that the algorithm sets $Z[i] = Z_i(A)$ for every $i \geq 2$.

Precondition	$A[1\dots n]$ is given; $l = r = 0$ before the loop.
Loop guard	i ranges from 2 to n.
Invariant	At the *start* of iteration i: (i) $Z[j]$ is correctly set for all $2 \leq j < i$, and (ii) $[l, r]$ is the Z-box with the largest right endpoint seen so far, i.e. $A[l\dots r] = A[1\dots r{-}l{+}1]$ (with $l = r = 0$ meaning no box yet).
Postcondition	$Z[i] = Z_i(A)$ for all $i \in \{2, \dots, n\}$.

Base case	Before $i = 2$: Z is empty and $l = r = 0$, so the invariant holds vacuously. ✓
Preservation	Assume the invariant holds at the start of iteration i. We analyse the two cases. *Case 1: $i > r$.* The while loop compares $A[1], A[2], \dots$ against $A[i], A[i{+}1], \dots$ directly and stops at the first mismatch, so $Z[i]$ is set to the correct value by definition. If $Z[i] > 0$, the box is updated to $[l, r] = [i,\, i{+}Z[i]{-}1]$, which has a larger r than before. ✓ *Case 2a: $i \leq r$ and $Z[k] < r{-}i{+}1$ (mirror strictly inside the box).* Let $k = i{-}l{+}1$. From the invariant, $A[l\dots r] = A[1\dots r{-}l{+}1]$, so $A[i{+}j] = A[k{+}j]$ for $j = 0, \dots, r{-}i$. In particular $A[i\dots i{+}Z[k]{-}1] = A[k\dots k{+}Z[k]{-}1] = A[1\dots Z[k]]$. Since $Z[k] < r{-}i{+}1$, position $k{+}Z[k]$ still lies inside $[1, r{-}l{+}1]$, so $A[i{+}Z[k]] = A[k{+}Z[k]] \neq A[1{+}Z[k]]$ (the character that ended the match at k). Therefore $Z[i] = Z[k]$. The box $[l, r]$ is not updated since $i{+}Z[i]{-}1 < r$. ✓ *Case 2b: $i \leq r$ and $Z[k] \geq r{-}i{+}1$ (mirror reaches or exceeds the box boundary).* The same translation gives $A[i\dots r] = A[1\dots r{-}i{+}1]$, so $Z[i] \geq r{-}i{+}1$ is guaranteed. The algorithm initialises $Z[i] = r{-}i{+}1$ and extends it by comparing $A[r{+}1], A[r{+}2], \dots$ directly until a mismatch, exactly as in Case 1. The box is then updated to $[i,\, i{+}Z[i]{-}1]$. ✓
Postcondition	When the loop exits, all $Z[i]$ have been correctly computed by the case analysis above. ✓

Amortised $\mathcal{O}(n)$ complexity. The variable r is non-decreasing and bounded by n, so it increments at most n times. Every comparison in the **while** loops either advances r (paid by the budget) or ends the step without advancing r (at most one per outer step i). Hence the total number of comparisons is $\mathcal{O}(n)$.

From Z values to *sp* values

The *sp* values of P can be read off directly from its Z-array.

Observation 1.1.20. $Z_j(P) = k$ means $P[j \dots j+k-1] = P[1 \dots k]$, so $P[1 \dots k]$ is a prefix-suffix of $P[1 \dots i]$ for $i = j+k-1$. We say j *maps to* i when $i = j + Z_j(P) - 1$.

Proposition 1.1.21. *For each* $i \in \{2, \dots, m\}$,

$$\mathrm{sp}_i(P) \;=\; Z_{j^*}(P), \qquad \textit{where} \quad j^* = \min\{\, j \geq 2 \mid j + Z_j(P) - 1 = i \,\}.$$

Proof. Any j mapping to i gives a prefix-suffix of $P[1 \dots i]$ of length $Z_j(P) = i - j + 1$. The minimum such j maximises $i - j + 1$, yielding the longest prefix-suffix, which is $\mathrm{sp}_i(P)$ by definition. □

Exercise 1.1.22. Using the Z-array of P, design an $\mathcal{O}(m)$ algorithm that computes $\mathrm{sp}_i(P)$ for every $i \in \{1, \dots, m\}$.

1.1.5 Strong borders and real-time KMP

The real-time bottleneck

KMP with plain *sp* values is *blind* (it never re-reads T) but not *real-time*: a single new character of T can trigger a fallback chain costing $\mathcal{O}(m)$ in the worst case. The fix is to collapse the entire chain into a single precomputed lookup. We achieve this in two logical steps: first by skipping strictly useless borders, and then by precomputing transitions for every possible character.

Strong border (sp'): Skipping useless fallbacks

Definition 1.1.23 (Strong border)**.** For $i \in \{1, \dots, m-1\}$, define

$$\mathrm{sp}_i(P)' \;=\; \max\{\, k \leq \mathrm{sp}_i(P) \mid k = 0 \text{ or } P[k+1] \neq P[i+1] \,\}.$$

This is the length of the longest proper prefix-suffix of $P[1 \dots i]$ whose next character differs from $P[i+1]$.

In plain words, $\mathrm{sp}_i(P)'$ is the length of the longest border of $P[1 \dots i]$ that would not cause an immediate mismatch at $P[i+1]$.

Any mismatch on $P[i+1]$ with a text character x would also fail at the next border $k = \mathrm{sp}_i(P)$ if $P[k+1] = P[i+1]$ — we say such borders are *useless* (see Figure 1.10). What is a useless border? Example 1.10 shows $P = \texttt{aabaa}$ at $i = 4$: the border $\alpha = \texttt{a}$ of length $k = 1$ is useless since $P[k+1] = P[2] = \texttt{a}$ equals $P[i+1] = P[5] = \texttt{a}$. If a mismatch occurs at $P[5]$ with some text character $x \neq \texttt{a}$, then after falling back to the border α, we would immediately fail again at $P[k+1]$ — the border is useless. sp' allows us to skip an entier chain of useless borders in a single step, jumping directly to the longest safe border that could potentially match the next text character.

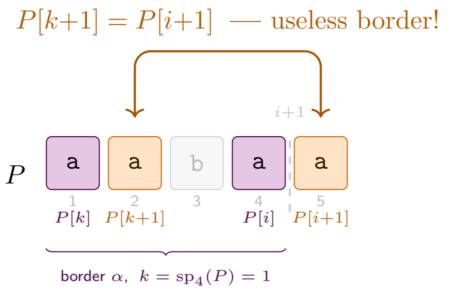


Figure 1.10: Strong border for $P =$ `aabaa` at $i = 4$. The border $\alpha =$ `a` (violet, $k = \mathrm{sp}_4(P) = 1$) satisfies $P[1] = P[4]$, but its "next character" $P[k{+}1] = P[2] =$ `a` (orange) equals $P[i{+}1] = P[5] =$ `a`. Any mismatch on $P[5]$ with text character $x \neq$ `a` would *also* fail immediately at $P[k{+}1]$ after the fallback — the border is useless. Hence $\mathrm{sp}_4(P)' = 0$: skip straight to the start.

$\mathrm{sp}_i(P)'$ skips the entire chain of useless borders in one step, but it only encodes a single fallback state — the longest "safe" border for the specific character $P[i+1]$. For a completely arbitrary text character x, a fallback chain might still occur. To achieve strict $\mathcal{O}(1)$ time per character, a full character-indexed table is needed.

Character-indexed borders and the DFA: True Real-Time

For a complete real-time solution we precompute, for each position i and character $x \in \Sigma$, the longest border of $P[1 \dots i]$ whose next character is exactly x:

Definition 1.1.24 (*$sp_{i,x}$* values)**.** For $i \in \{0, 1, \dots, m\}$ and $x \in \Sigma$,

$$\mathrm{sp}_{i,x}(P) \;=\; \max\{\, k \leq \mathrm{sp}_i(P) \mid P[k+1] = x \,\},$$

with default 0 when no such k exists.

This generalises $\mathrm{sp}_i(P)'$: where the strong border encodes a single fallback state for the character $P[i+1]$, the table $\mathrm{sp}_{i,x}$ encodes the optimal fallback for *every* $x \in \Sigma$ simultaneously.

Example 1.1.25 (*sp'* vs. $\mathrm{sp}_{i,x}$ for $P =$ `aabaab`)**.** Consider $P =$ `aabaab` ($m = 6$) over $\Sigma = \{$`a`, `b`, `c`$\}$. The *sp* and *sp′* values are:

i	1	2	3	4	5	6
$P[i]$	`a`	`a`	`b`	`a`	`a`	`b`
$\mathrm{sp}_i(P)$	0	1	0	1	2	3
$\mathrm{sp}_i(P)'$	0	1	0	0	1	–

At $i = 5$ the border `aa` ($\mathrm{sp}_5(P) = 2$) is useless because $P[3] =$ `b` $= P[6]$; the shorter border `a` ($k = 1$) satisfies $P[2] =$ `a` $\neq P[6]$, so $\mathrm{sp}_5(P)' = 1$.

At $i = 5$, the border chain of $P[1..5] =$ `aabaa` is $k \in \{2, 1, 0\}$. Both $k = 2$ ($P[3]$=`b`=$P[6]$) and $k = 1$ ($P[2]$=`a`) must be inspected; $\mathrm{sp}_5(P)' = 1$ skips past the useless $k = 2$.

Now suppose we have matched $P[1..5]$ (state $j = 5$) and the next text character is x. With $\mathrm{sp}_5(P)' = 1$, KMP falls back to state 1 for *every* mismatch $x \neq P[6] =$ `b`, and may still need further comparisons depending on x:

$\mathrm{sp}_{5,\mathtt{a}} = 1$ because the border of length 1 satisfies $P[2] = \mathtt{a} = x$; no border has $P[k+1] = \mathtt{c}$, so $\mathrm{sp}_{5,\mathtt{c}} = 0$.

- $x = \mathtt{a}$: $5 \to 1$, then $P[2] = \mathtt{a} = x \Rightarrow$ state 2. **Two comparisons.**
- $x = \mathtt{c}$: $5 \to 1$, $P[2] \neq \mathtt{c}$; $1 \to 0$, $P[1] \neq \mathtt{c}$; stay at 0. **Three comparisons.**

By contrast, $\mathrm{sp}_{5,x}$ resolves every case in one lookup:

x	a	b	c
$\mathrm{sp}_{5,x}$	1	– (match $P[6]$)	0
DFA $\delta(5,x)$	2	6 (full match!)	0

$\delta(5,\mathtt{a}) = 2$ replaces two comparisons with one table read; $\delta(5,\mathtt{c}) = 0$ replaces three.

The DFA has $m+1$ states and $|\Sigma|$ transitions per state, so the table takes $\mathcal{O}(m|\Sigma|)$ space and time to build.

The table $\{\mathrm{sp}_{i,x}(P)\}_{i,x}$ is precisely the *transition function* of the deterministic finite automaton (DFA) for pattern P: state i encodes "i characters of P matched so far", and on reading x the automaton transitions to $\mathrm{sp}_{i,x}(P) + \mathbf{1}[P[i+1] = x]$. With this table, each character of T is processed in strict $\mathcal{O}(1)$ worst-case time, making KMP truly **real-time**.

1.1.6 A Comprehensive Trace: KMP and Z-Algorithm

To solidify these concepts, let's trace KMP and then contrast it directly with the Z-Algorithm.

KMP Step-by-Step

Consider the pattern $P = \mathtt{ababa}$ and the text $T = \mathtt{ababcababa}$. Here we trace the standard KMP behavior (using plain *sp* values to illustrate the fallback mechanism that DFA optimizes).

For $i = 4$, $P[1..4] = \mathtt{abab}$. Proper prefix-suffixes are $\mathtt{ab}$ (len 2) and $\mathtt{a}$ (len 1). Max is 2.

1. Preprocessing P. We compute the *sp* array for P.

i	1	2	3	4	5
$P[i]$	a	b	a	b	a
$\mathrm{sp}_i(P)$	0	0	1	2	3

2. Searching T. We maintain a pointer i into T and a pointer j representing the length of the current match in P. The complete trace is shown in Figure 1.11.

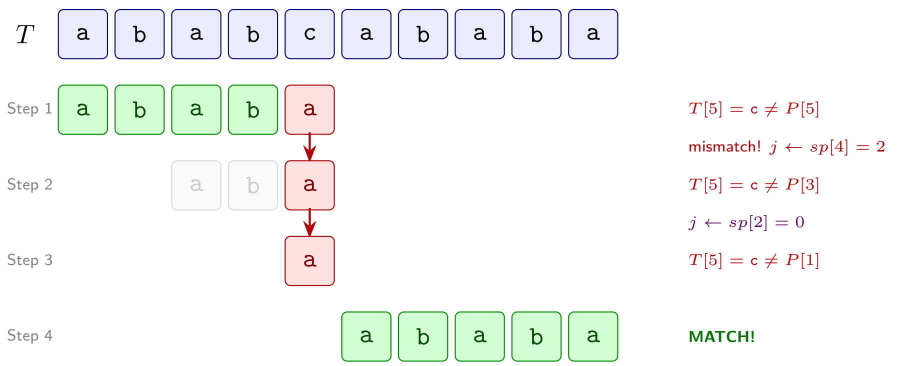


Figure 1.11: KMP trace for $P = \texttt{ababa}$ in $T = \texttt{ababcababa}$. Step 1 finds a mismatch at $T[5]$. KMP falls back twice ($j = 4 \to 2 \to 0$) without re-reading T. Finally, Step 4 finds a complete match starting at $T[6]$.

Online vs. Real-Time: Bridging the Gap

The KMP trace above highlights a crucial property: KMP processes T one character at a time without ever stepping backwards in T. This makes KMP an **online** algorithm. However, as seen in Steps 1-3, processing the single character $T[5] = \texttt{c}$ required multiple internal operations (j falling back multiple times). If we had used the full DFA ($\mathrm{sp}_{i,x}$) discussed previously, those steps would collapse into $\mathcal{O}(1)$ worst-case time, upgrading the algorithm from merely online to strictly **real-time**.

By contrast, the Z-algorithm is *neither* real-time nor strictly online in the same way. The computation of the Z-box $[l, r]$ depends on reading characters ahead in the string. Let's trace it to see this structural difference in action.

Z-Algorithm Trace

Let's compute the Z-values for the string $S = \texttt{aababca}$. Notice how the algorithm looks ahead to establish the rightmost boundary r.

1. **$i = 2$:** $S[2] = \texttt{a} = S[1]$. Check $S[3] = \texttt{b} \neq S[2]$. $Z_2 = 1$. Box $[l, r] = [2, 2]$.
2. **$i = 3$:** $i > r = 2$. $S[3] = \texttt{b} \neq S[1]$. $Z_3 = 0$. Box stays $[2, 2]$.
3. **$i = 4$:** $i > r = 2$. $S[4] = \texttt{a} = S[1]$. Check $S[5] = \texttt{b} = S[2]$, $S[6] = \texttt{c} \neq S[3]$. $Z_4 = 2$. New box $[l, r] = [4, 5]$.
4. **$i = 5$:** $i \leq r = 5$. Mirror $k = i - l + 1 = 5 - 4 + 1 = 2$. $Z_k = Z_2 = 1$. Remaining box width $r - i + 1 = 5 - 5 + 1 = 1$. Since $Z_k \not< r - i + 1$ (Case 2b), we try to extend from $r + 1 = 6$. $S[6] = \texttt{c} \neq S[3]$. No extension. $Z_5 = 1 + 0 = 1$. Box stays $[4, 5]$.
5. **$i = 6$:** $i > r = 5$. $S[6] = \texttt{c} \neq S[1]$. $Z_6 = 0$.
6. **$i = 7$:** $i > r = 5$. $S[7] = \texttt{a} = S[1]$. No more chars. $Z_7 = 1$.

i	1	2	3	4	5	6	7
$S[i]$	a	a	b	a	b	c	a
Z_i	–	1	0	2	1	0	1

1.1.7 Boyer-Moore Algorithm

So far we have seen algorithms that scan the text strictly left to right, never revisiting a character once processed. The Boyer-Moore algorithm takes a different approach: it compares the pattern against the text *from right to left* while still sliding the pattern from left to right. Mismatches on the rightmost characters of the pattern are the cheapest failures possible—they let the algorithm skip large portions of the text.

Scanning right-to-left is the key insight: a mismatch at the last *character of the current alignment gives the most information about how far to jump.*

Intuition

The algorithm combines two independent heuristics. At every alignment it takes the *maximum* shift allowed by either rule:

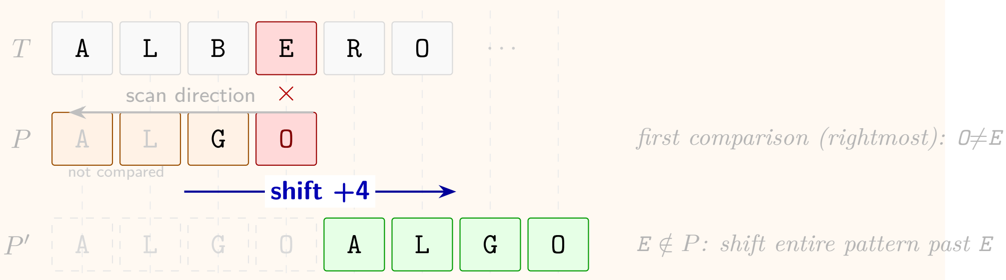
- **Bad Character Rule (BCR).** When character x in the text causes a mismatch at position i in the pattern, shift P right so that the rightmost occurrence of x in $P[1 \dots i{-}1]$ aligns with x in the text. If x does not appear in P at all, skip past x entirely.
- **Good Suffix Rule (GSR).** When the suffix $S = P[i \dots m]$ has already been matched and position $i{-}1$ causes a mismatch, shift P right so that another copy of S in P—preceded by a character *different* from $P[i{-}1]$—aligns with the matched copy of S in the text.

Both rules are correct in the sense that neither one ever skips a genuine occurrence of P. At each step Boyer-Moore applies whichever rule yields the larger shift.

Bad Character Rule

Definition 1.1.26 (Bad Character table)**.** For each position $1 \le i \le m$ and character $x \in \Sigma$, let $\mathrm{BC}(i, x)$ be the largest $j < i$ such that $P[j] = x$, or 0 if no such j exists. When a mismatch occurs at $P[i]$ against text character x, shift P right by $i - \mathrm{BC}(i, x)$ positions.

Example 1.1.27 (BCR skip)**.** Search $P =$ `ALGO` in $T =$ `...ALBERO...`. Aligning P with the first four characters, we compare right-to-left: $P[4] =$ `O` vs $T[4] =$ `E` is a mismatch. Since `E` does not appear in P, BCR shifts the entire pattern past the "bad" character.



Preprocessing options. Depending on the alphabet size $|\Sigma|$ and pattern length m, different data structures can be used to implement the Bad Character Rule:

a) **Full 2-D table** ($|\Sigma| \times m$): A matrix $M[x, i]$ storing $\mathrm{BC}(i, x)$. This provides $\mathcal{O}(1)$ query time but requires $\mathcal{O}(|\Sigma|m)$ space, which is only feasible for small alphabets like DNA.

b) **Compressed Matrix (Array of Lists):** For each position $1 \le i \le m$, store a linked list of pairs (x, j) where j is the rightmost occurrence of character x in $P[1 \dots i-1]$. This is a sparse representation of the 2-D table (see Figure 1.12). The space complexity is $\mathcal{O}(m \cdot \min(m, |\Sigma|))$, and the query time is $\mathcal{O}(\min(m, |\Sigma|))$.

c) **Position Lists (Character-indexed lists):** For each character x that appears in P, store a list of its occurrence indices in P in *descending* order (see Figure 1.13). To find $\mathrm{BC}(i, x)$, we search the list for x for the largest index $j < i$. Space is $\mathcal{O}(m)$, and query time is $\mathcal{O}(\log m)$ with binary search.

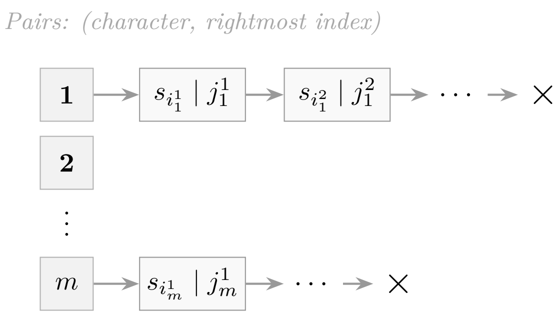


Figure 1.12: BCR as an array of $|P|$ linked lists (Sparse Matrix representation).

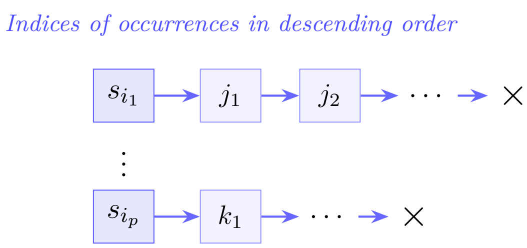


Figure 1.13: BCR as an array of character-indexed lists.

> *Remark* 1.1.28. While binary search on position lists costs $\mathcal{O}(\log m)$, the total pattern length m is typically small enough that this overhead is negligible compared to the large shifts BCR provides.

Good Suffix Rule

Intuition. The Bad Character Rule ignores the characters we *already matched*. When Boyer-Moore scans right-to-left and a mismatch occurs at position i, we have successfully matched the suffix $S = P[i \dots m]$ against the text. That information is precious: if S appears again somewhere to the left in P, we can shift P so that second occurrence lines up with the matched region in T and continue — no re-reading of T required.

> **Example 1.1.29** (GSR intuition)**.** Let $P =$ `CABAB` and suppose scanning right-to-left we match $S = P[4 \dots 5] =$ `AB` against T, then mismatch at $P[3] =$ `B`. The substring `AB` also appears in P at positions 2–3, preceded by `C` $\neq$ `B` (so the same mismatch cannot immediately recur). We shift P right by 2 to realign that copy of `AB` with the matched region of T.

The Good Suffix Rule is the Boyer-Moore analogue of KMP's border table: both exploit internal repetition of the pattern to skip shifts that are guaranteed to fail.

Two boundary situations arise:

- If no copy of S exists inside P (other than at the end), we fall back to the longest *prefix* of P that is a suffix of S, and align that prefix with the end of the matched region.
- If even that fails (no prefix of P matches any suffix of S), we shift the entire pattern past the matched region.

After matching suffix $S = P[i \dots m]$, we want to shift P so that another copy of S aligns with the text. We require that copy to be preceded by a character *different* from $P[i-1]$; otherwise the same mismatch would recur immediately.

> **Definition 1.1.30** ($L'(i)$)**.** $L'(i)$ is the largest index $j < m$ such that
>
> 1. $P[i \dots m]$ occurs in P ending at position j, and
> 2. the character immediately before that occurrence differs from $P[i-1]$: $P[j - (m - i)] \neq P[i-1]$.

Figure 1.14 illustrates the geometry: the suffix $S = P[i \dots m]$ appears again ending at position $L'(i)$, and the character immediately to its left (x) differs from $y = P[i-1]$, guaranteeing the shift avoids an immediate re-mismatch.

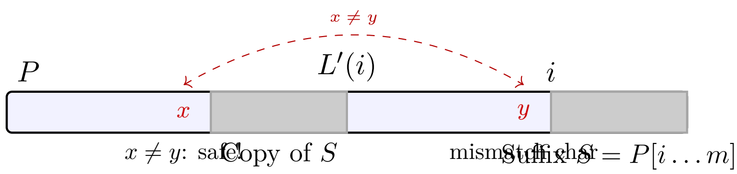


Figure 1.14: Geometry of $L'(i)$. The suffix $S = P[i \dots m]$ occurs again ending at $L'(i)$, preceded by a character $x \neq y = P[i-1]$. Shifting P so that the copy of S aligns with the matched text region guarantees the mismatch character changes, so the same failure cannot repeat.

To compute $L'(i)$ efficiently we need a helper quantity that measures how much of P's own suffix repeats inside P. Intuitively, $L'(i) = j$ means that $P[j - (m - i) \dots j] = P[i \dots m]$, so we need to find the rightmost position j where such a match ends. That is exactly captured by the N-*values*:

> **Definition 1.1.31** ($N_j(P)$)**.** $N_j(P)$ is the length of the longest suffix of $P[1 \dots j]$ that is also a suffix of P.

Theorem 1.1.32. *The values $N_j(P)$ satisfy*

$$N_j(P) = Z_{m-j+1}(P^r),$$

where $m = |P|$ and P^r denotes P reversed. Consequently, the entire GSR table can be computed in $\mathcal{O}(m)$ time by running the Z-algorithm on P^r.

Formal details — **Proof:** $N_j(P) = Z_{m-j+1}(P^r)$

Set $k = m - j + 1$ and recall the two definitions:

- $N_j(P) = \ell$ iff ℓ is the largest integer such that $P[j-\ell+1 \ldots j] = P[m-\ell+1 \ldots m]$, i.e. the last ℓ characters of $P[1 \ldots j]$ equal the last ℓ characters of P.
- $Z_k(P^r) = \ell$ iff the length-ℓ prefix of P^r matches P^r at position k: $P^r[1 \ldots \ell] = P^r[k \ldots k+\ell-1]$.

Key observation. Since $P^r[i] = P[m + 1 - i]$, reading P^r from left to right is the same as reading P from right to left. More precisely, for any $\ell \geq 1$ and $s = 0, 1, \ldots, \ell - 1$:

$$P^r[1 + s] = P[m - s] \quad \text{and} \quad P^r[k + s] = P[m + 1 - k - s] = P[j - s],$$

where the last equality uses $k = m - j + 1$.

The two conditions are equivalent. For any $\ell \geq 0$:

$$\begin{aligned} P^r[1 \ldots \ell] = P^r[k \ldots k+\ell-1] &\iff P[m-s] = P[j-s] \; \forall\, s \in \{0, \ldots, \ell-1\} \\ &\iff P[m-\ell+1 \ldots m] = P[j-\ell+1 \ldots j]. \end{aligned}$$

The rightmost condition is exactly the requirement that $P[1 \ldots j]$ has a suffix of length ℓ that matches a suffix of P.

Conclusion. Because the two conditions agree for every ℓ, their maxima agree:

$$N_j(P) = Z_{m-j+1}(P^r).\square$$

Complexity. Running the Z-algorithm on P^r costs $\mathcal{O}(m)$ time and produces all m values $Z_1(P^r), \ldots, Z_m(P^r)$ in a single pass. Reading off $N_j = Z_{m-j+1}$ for $j = 1, \ldots, m$ is then an $\mathcal{O}(m)$ post-processing step, so the full GSR precomputation is $\mathcal{O}(m)$ overall.

Example 1.1.33 (N_j and $L'(i)$ on a concrete pattern)**.** Let $P =$ `CABAB` ($m = 5$). The reversed pattern is $P^r =$ `BABAC`. Running the Z-algorithm on P^r:

position k	1	2	3	4	5
$P^r[k]$	B	A	B	A	C
$Z_k(P^r)$	–	0	2	0	0

Converting via $N_j = Z_{m-j+1}(P^r)$:

- $N_5 = Z_1 =$ (by convention, the full string) $= 5$
- $N_4 = Z_2 = 0$
- $N_3 = Z_3 = 2$ (suffix `AB` of $P[1\ldots3] =$ `CAB` matches suffix `AB` of P)
- $N_2 = Z_4 = 0$
- $N_1 = Z_5 = 0$

From $N_3 = 2$ we read: the last 2 characters of $P[1\ldots3]$ form a suffix of P, i.e. $P[2\ldots3] = P[4\ldots5] =$ `AB`. This directly gives $L'(4) = 3$: the rightmost occurrence of $P[4\ldots5]$ inside P ending strictly before m ends at position 3, preceded by $P[1] =$ `C` $\neq P[3] =$ `B`. GSR shift for a mismatch at position 4: $m - L'(4) = 5 - 3 = 2$.

Reusing the Z-algorithm on P^r means no new code is needed for GSR preprocessing: run the same routine on the reversed pattern.

Figure 1.15 shows the reversal correspondence between $Z_k(P^r)$ and $N_j(P)$ visually: a match of length ℓ starting at position k in P^r corresponds exactly to a suffix-match of length ℓ ending at position j in P.

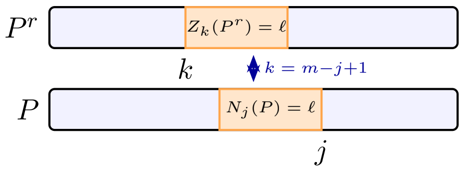


Figure 1.15: Reversal correspondence: $N_j(P) = Z_{m-j+1}(P^r)$. A Z-match of length ℓ at position k in the reversed pattern corresponds to a suffix-match of length ℓ ending at position $j = m - k + 1$ in the original pattern.

Algorithm Implementation

We now have all the ingredients. Both rules are precomputed in $\mathcal{O}(m)$ time: BCR uses the bad-character table $\mathrm{BC}(i, x)$ (position lists or 2-D table), and GSR uses the N_j values derived from the Z-algorithm on P^r. At each alignment, Boyer-Moore scans right-to-left, and on a mismatch at position j applies *both* rules independently, then takes the **maximum** shift — this is safe because each rule on its own guarantees not to skip any occurrence.

The full Boyer-Moore algorithm combines the precomputed Bad Character and Good Suffix tables to determine the maximum possible shift at each step.

Algorithm 5: Boyer-Moore Pattern Matching

Input: Text T of length n, Pattern P of length m

Output: Indices of all occurrences of P in T

```
BC ← PrecomputeBadCharacter(P)
GS ← PrecomputeGoodSuffix(P)
s ← 0                                   // Current shift
while s ≤ n − m do
    j ← m
    while j > 0 and P[j] = T[s + j] do
        j ← j − 1
    if j = 0 then
        Report occurrence at s + 1
        s ← s + GS[1]
    else
        shift_BC ← max(1, j − BC(j, T[s + j]))
        shift_GS ← GS[j]
        s ← s + max(shift_BC, shift_GS)
```

Complexity

> **Theorem 1.1.34** (Boyer-Moore worst case)**.** *In the worst case, Boyer-Moore runs in $\mathcal{O}(nm)$ time.*

Proof sketch. Consider $T = a^n$ and $P = ba^{m-1}$. At every alignment the algorithm compares all m characters right-to-left (matching $m-1$ `a`'s before mismatching on `b`), and both BCR and GSR produce a shift of only 1. There are $n - m + 1$ alignments, giving $(n - m + 1) \cdot m = \mathcal{O}(nm)$ comparisons. □

> *Remark* 1.1.35 (Average-case behaviour)*.* For a random text over a large alphabet, BCR almost always triggers a skip at the very first (rightmost) comparison, since a random character is unlikely to appear in P. The expected shift is $\mathcal{O}(m)$, leading to an average-case complexity of $\mathcal{O}(n/m)$ — sub-linear in n, which is why Boyer-Moore is so effective in practice for natural-language or genomic texts.

Apostolico–Giancarlo Optimization

Intuition. Boyer-Moore's $\mathcal{O}(nm)$ worst case stems from *forgetting*: after each alignment it discards all information about which text characters were compared, so the same characters can be re-examined from scratch at the next shift. This is the same problem KMP solved for single-pattern matching — and the fix is the same idea: *cache* the outcome of previous comparisons.

Apostolico and Giancarlo maintain an auxiliary array M over the text. After each alignment, whenever a suffix of P of length ℓ is matched against T ending at position k, they store $M[k] = \ell$. At a future alignment, before comparing $P[i]$ against $T[k]$, the algorithm checks whether $M[k]$ already records how much of the pattern matches at k — and if so, skips those comparisons entirely.

Definition 1.1.36 (M array)**.** $M[k]$ is the length of the longest suffix of P that was matched against T ending at position k in some previous alignment. Initially $M[k] = 0$ for all k.

When P is aligned at shift s and we are comparing $P[i]$ with $T[k]$ (where $k = s + i$), we check $w = M[k]$. Recall that $N_i(P)$ is the length of the longest suffix of $P[1 \ldots i]$ that is also a suffix of P.

There are four key cases when we encounter a previously matched segment ($w > 0$):

1. **Case 1: $w > N_i$.** Since the text matches a longer suffix of P than what is available as a sub-occurrence ending at i, there must be a mismatch at $i - N_i$. We can immediately shift.

2. **Case 2: $w < N_i$.** The text matches a shorter suffix than the available sub-occurrence. This implies a mismatch at $i - w$.

3. **Case 3: $w = N_i$.** The match in the text exactly matches the sub-occurrence of the suffix in P. We can skip w comparisons and continue checking P from position $i - w$.

4. **Case 4: $w \geq N_i$ and $N_i = i$.** This corresponds to a prefix of P matching the text. If $i = m$, we found a full match; otherwise, we can skip and continue.

Figure 1.16 summarises the four cases geometrically.

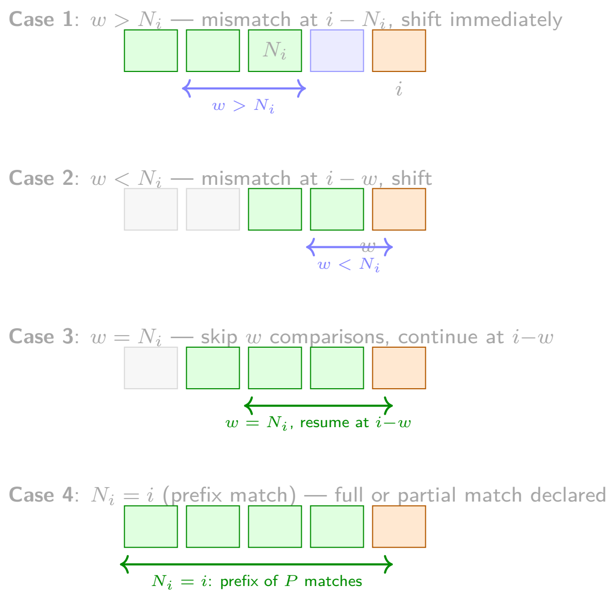


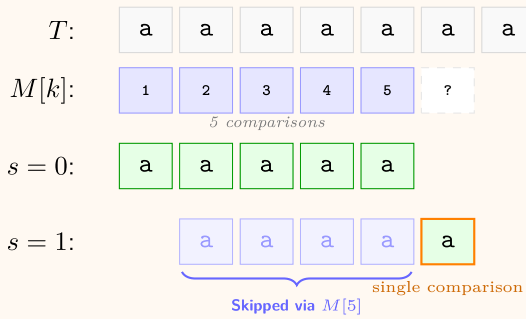
Figure 1.16: The four cases of the Apostolico–Giancarlo algorithm. Green cells are already known to match (from $M[k]$); blue cells are skipped; orange is the current comparison position i.

By using these rules, Apostolico–Giancarlo ensures that each character of the text is involved in at most a constant number of *successful* comparisons, bringing the worst-case complexity down to $\mathcal{O}(n + m)$.

Example 1.1.37 (Apostolico–Giancarlo in action)**.** Consider the classic worst-case for Boyer-Moore: $P = \texttt{aaaaa}$ $(m = 5)$ and $T = \dots\texttt{aaaaaa}\dots$. Precomputing $N_i(P)$: since every prefix of P is also a suffix of P, we have $N_i = i$ for all i.

1. **Shift** $s = 0$: Compare P against $T[1\dots5]$ right-to-left. 5 matches found. Update array M with match lengths: $M[1] = 1, M[2] = 2, M[3] = 3, M[4] = 4, M[5] = 5$.

2. **Shift** $s = 1$: Align P starting at $T[2]$.
   - Compare $P[5]$ with $T[6]$. Match! (1 comparison).
   - Move to $P[4]$ vs $T[5]$. We see $M[5] = 5$ (previous match).
   - Check Case 4: $w = 5 \geq N_4 = 4$ and $N_4 = 4$ (prefix match).
   - **Jump**: We can declare a full match for the rest of the pattern without any further comparisons.



In this scenario, while Boyer-Moore would perform m comparisons for each shift ($\mathcal{O}(nm)$), AG performs only one (plus the M check), guaranteeing $\mathcal{O}(n)$ linearity even on periodic strings.

On the other hand, in favourable inputs (e.g. $T = c^n$, $P = a^{m-1}c$) the original Boyer-Moore already scans the text in as few as $\lceil n/m \rceil$ character comparisons.

1.1.8 Generalization of the Problem: Multiple Patterns

The algorithms we have seen so far are designed to search for a single pattern P in a text T. In many applications, we need to search for multiple patterns simultaneously. For example, in DNA sequencing, we might want to find all occurrences of a set of motifs in a long genome sequence.

Definition 1.1.38 (Multiple Pattern Matching)**.** Given a set of patterns $\mathcal{P} = \{P_1, P_2, \dots, P_z\}$ and a text T, find all occurrences of all pattern in $\mathcal{P}$ within T.

Naïve Approach

As usual, let's establish a baseline to compare against. The simplest idea is to run a single-pattern algorithm (e.g. KMP or the Z-algorithm) once for *each* pattern in $\mathcal{P}$ independently.

Denote $m_i = |P_i|$ and let $m = \sum_{i=1}^{z} m_i$ be the total pattern length. Each invocation costs $\mathcal{O}(n + m_i)$, so the overall cost is

$$\mathcal{O}(n + m_1) + \cdots + \mathcal{O}(n + m_z) \;=\; \mathcal{O}(z \cdot n + m)\,.$$

The $z \cdot n$ term dominates when many patterns are present: we are re-reading T once per pattern, ignoring any shared structure among the P_i.

The culprit is the factor z multiplying n: we scan T from scratch for every pattern, wasting all the information accumulated from previous passes.

Can we do better? Ideally we would like to scan T only *once*, achieving $\mathcal{O}(n + m)$ total time regardless of z. The key insight is that instead of processing the patterns independently, we should *combine* all of them into a single shared data structure, then query it while performing a single left-to-right pass over T.

The natural structure for organising a set of strings by their shared prefixes is a tree—specifically, a **keyword tree**.

Keyword Tree

Definition 1.1.39 (Keyword tree, $K(\mathcal{P})$)**.** Given a set of patterns $\mathcal{P} = \{P_1, \ldots, P_z\}$ over alphabet Σ, the **keyword tree** $K(\mathcal{P})$ is a rooted tree whose edges are labelled with characters of Σ such that:

(i) edges leaving the same node carry *distinct* labels;

(ii) for each $i \in \{1, \ldots, z\}$ there is exactly one root-to-leaf path whose edge-labels, read in order, spell P_i; conversely, every root-to-leaf path spells some $P_i \in \mathcal{P}$.

Remark 1.1.40 (Simplifying assumption)*.* Throughout this section we assume no P_i is a proper prefix of another P_j. Under this assumption every pattern ends at a *leaf*, giving a **bijection** between leaves and patterns. The Aho–Corasick construction lifts this restriction.

The bijection means: a single pointer inside $K(\mathcal{P})$ simultaneously represents all *patterns that share the prefix corresponding to the current node — the exact analogue of the single state in KMP, now for a whole set of patterns.*

Observation 1.1.41 (Size, height, and fan-out of $K(\mathcal{P})$).

- **Size.** The number of edges is at most $m = \sum_{i=1}^{z} m_i$: every non-root node is introduced by a distinct edge, and each root-to-leaf path contributes exactly m_i edges.
- **Height.** The longest root-to-leaf path has length $m_{\max} = \max_i m_i$, the length of the longest pattern.
- **Fan-out.** By property (i), the children of any node carry distinct labels, so the fan-out is at most $|\Sigma|$.

Example 1.1.42 (Building a keyword tree)**.** Let $\mathcal{P} = \{\texttt{potato}, \texttt{tatoo}, \texttt{theater}, \texttt{other}\}$. We insert the patterns one by one from the root, sharing edges as long as prefixes agree:

- `potato` and no other pattern share a prefix → a chain of 6 edges labelled `p-o-t-a-t-o` from the root.
- `tatoo` starts with `t`; no existing edge from the root is labelled `t` → a new branch. At depth 1 the single `t`-node branches further for `a`

(continuing `tatoo`) and later for `h` (from `theater`).

- `theater` shares the prefix `t` with `tatoo`; at depth 1 they diverge (`a` vs `h`).
- `other` starts with `o`; no existing root-edge is labelled `o` $\to$ a new branch.

The resulting tree has 6+5+7+5 = 23 edges (one per character of all patterns).

Figure 1.17 shows the keyword tree for $\mathcal{P} = \{$`potato`, `tatoo`, `theater`, `other`$\}$. The `t`-subtree branches at depth 2 into the `a`-path (for `tatoo`) and the `h`-path (for `theater`), reflecting the shared prefix `t`.

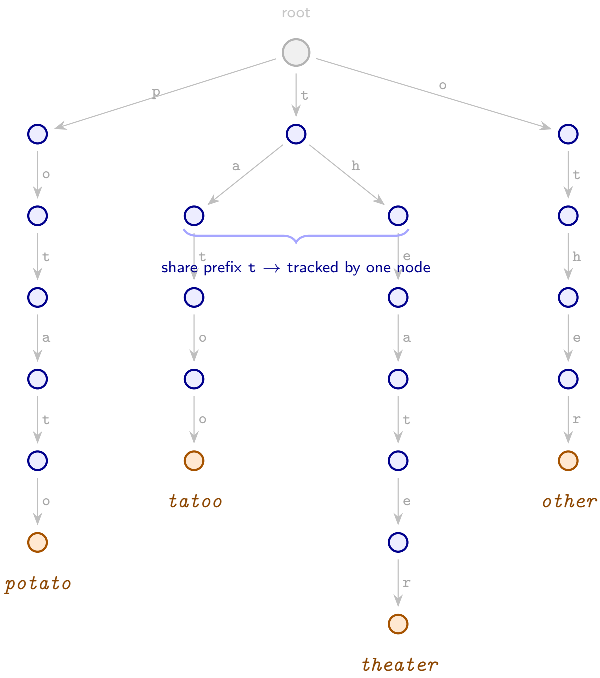


Figure 1.17: Keyword tree $K(\mathcal{P})$ for $\mathcal{P} = \{$`potato`, `tatoo`, `theater`, `other`$\}$. Grey root, blue internal nodes, orange leaves. The `tatoo` and `theater` sub-trees branch from the same `t`-node (depth 1), reflecting their shared first character.

Using $K(\mathcal{P})$ on T

The tree is built once in $\mathcal{O}(m)$ time. We then make a *single* left-to-right pass over T: at each starting position s we descend $K(\mathcal{P})$ from the root following edge-labels $T[s], T[s+1], \ldots$; reaching a leaf at depth m_i reports an occurrence of P_i ending at $T[s+m_i-1]$.

The key gain: all patterns sharing a common prefix are tracked in parallel *by a single internal node. As long as the text characters keep matching a shared prefix, no extra work is done for any of them — the tree compresses the joint prefix-matching state for the whole set $\mathcal{P}$.*

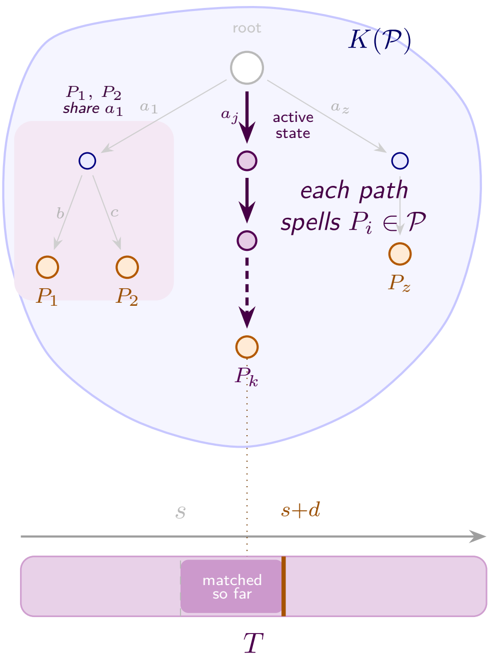


Figure 1.18: Scanning T with $K(\mathcal{P})$. At each starting position s we follow edge-labels $T[s], T[s+1], \ldots$ downward from the root (depth d reached so far is shaded on T); reaching a leaf reports an occurrence of the corresponding pattern. Patterns with a shared prefix — here P_1 and P_2 both start with a_1 (pale violet zone) — are tracked in parallel by a single internal node, so no extra work is done for either until they diverge. The highlighted violet path is the *active* descent for the current starting position.

This already beats the naïve approach when patterns share long prefixes, but the worst-case cost per starting position is still $\mathcal{O}(m_{\max})$, giving $\mathcal{O}(n \cdot m_{\max} + m)$ overall.

Why the keyword tree alone is not enough. Consider searching for $\mathcal{P} = \{\texttt{he}, \texttt{she}, \texttt{his}, \texttt{hers}\}$ in $T = \texttt{ushers}$. After reading `u` from the root we fail immediately (no `u`-edge), restart from the root, try `s` $\to$ `h` $\to$ `e` $\to$ leaf (`she` found). Now the *naive* tree traversal restarts from the root, but the `e` we just read is itself the start of `he`! We missed a match because we threw away the suffix information.

This is exactly the problem KMP solved for a single pattern with the sp' table: instead of restarting from scratch, jump to the longest proper border of the current prefix and continue from there. For multiple patterns the same idea generalises: at a mismatch (or after a full match) we follow a **failure link** from the current node to the node representing the longest proper suffix of the current prefix that is also a prefix of some pattern in $\mathcal{P}$. These failure links turn $K(\mathcal{P})$ into a finite automaton that processes T in a single left-to-right pass, never re-reading any character.

> **Definition 1.1.43** (Failure link)**.** For a node v in $K(\mathcal{P})$ representing a string α, the **failure link** of v points to the node representing the longest proper suffix of α that is a prefix of some pattern in $\mathcal{P}$ (or to the root if no such suffix exists).

Failure links in $K(\mathcal{P})$ are the direct multi-pattern generalisation of the sp' values in KMP: both encode "the longest previously matched prefix you can fall back to".

To achieve a true $\mathcal{O}(n+m)$ single pass we need to handle mismatches without restarting from the root — the same challenge KMP solved for a single pattern. The solution is to equip $K(\mathcal{P})$ with failure links, leading to the **Aho–Corasick** automaton.

Chapter 2

Randomized Algorithms

Chapter 3

Data Compression